

Jun 9, 2026

Section 1: Technical Understanding

The most surprising discovery in this lab was how completely and literally an image dissolves into numbers. To understand intellectually beforehand that digital images are stored as data. Then at the same time watch a brightness adjustment reduce to a simple NumPy addition. Adding an integer to every element of a matrix made the concept visceral rather than abstract. There is no color, no shape, no meaning stored anywhere in the file, only arrays of integers.

Everything a human perceives when looking at a photograph is reconstructed entirely in the act of display, from raw numeric data like matrix mega pixels. That reduction is both elegant and humbling. The relationship between mathematical operations and visual effects proved to be unexpectedly straightforward. A convolution kernel is just a small matrix of weights. Yet sliding it across the image produces effects as distinct as smoothing, edge sharpening, or embossing. Depending solely on which numbers fill the 3×3 grid. The sharpening kernel, for example, places a large positive value at the center and negative values at the neighbors. Which mathematically amplifies differences and visually produces crisper edges. Once you see that correspondence, every visual effect you have ever seen in a photo editing application starts to feel tractable rather than magical. The most challenging technique to understand was histogram equalization. The intuition spreading pixel values to use the full 0–255 range is clear enough, but the cumulative distribution function mapping that implementing it requires careful thought. The CDF essentially reassigns each pixel's value based on how many pixels fall below it. This guarantees an uniform distribution in the

output. Understanding why this works (and why CLAHE limits it to local regions to prevent over-enhancement) required reading the code carefully alongside the histogram visualizations rather than just running it.

Section 2: Connections and Applications

The connection between this lab and the Nano Banana demonstration from class became clear during Part 5. Every operation we implemented by hand. The convolution, edge detection, color channel manipulation is precisely what a convolutional neural network does. Except that in a CNN the kernel values are not written by a human but learned from training data. When Nano Banana processes an image, it runs the equivalent of thousands of our small kernels. In such a way that stacked into layers, each discovering progressively more complex patterns. The blur kernel we wrote is a single learned filter in the first layer of a real CNN. The mathematics are identical; only the scale and the origin of the weights differ. This realization changed how I watch AI image tools work. I will no longer see them as opaque black boxes but very large. Automated versions of what we did manually.

The real-world applications of these techniques are broader than I initially appreciated. Medical imaging relies on histogram equalization and edge detection to make subtle features visible in X-rays and MRIs. Autonomous vehicles use Canny edge detection and perspective transformation for lane detection and obstacle recognition. Smartphone cameras apply unsharp masking and local contrast enhancement invisibly every time a photo is taken. Satellite imagery analysts use geometric correction (essentially the perspective transformation we implemented) to rectify images captured at oblique angles. Even the Instagram filter pipeline we built from saturation boosting. Vignette darkening mirrors what commercial filter apps do at scale.

In a future project, I would combine traditional and AI approaches by using classical preprocessing to normalize and clean input images before feeding them to a neural network.

For example, applying CLAHE to equalize contrast across a dataset of medical images before training a diagnostic classifier would remove a major source of variability that the network would otherwise need to learn to handle. Traditional operations are deterministic, interpretable, and fast. Ideally for preprocessing while AI excels at the high-level pattern recognition task that follows.

Section 3: Personal Reflection

The aspect of image processing I find most compelling for further exploration is Fourier-domain filtering. Everything we implemented in this lab operates in the spatial domain, we slide a kernel across pixels. The Fourier transform converts an image into frequency space. Whereas operations like removing repetitive noise patterns or isolating textures at specific scales become straightforward arithmetic rather than complex spatial computations. Understanding this connection between spatial and frequency representations would also deepen my understanding of why CNNs work the way they do. Spectral analysis has been used to explain what deep networks are learning.

This lab substantially changed my understanding of digital photography. I previously thought of photo editing as a kind of artistry disconnected from mathematics. Adjusting sliders until something looks right. I now understand that every slider in every photo editing application maps directly to a mathematical operation on a matrix. The exposure slider adds a constant, the clarity slider applies an unsharp mask, the vignette draws a radial gradient and multiplies. That transparency makes the tools feel more like instruments I understand rather than control what I manipulate by intuition.

Questions I still have center on the limits of classical techniques relative to AI. Specifically, for what categories of tasks does a carefully engineered classical pipeline still outperform a trained CNN. In terms of accuracy, speed, or interpretability and where does the advantage definitively shift to learned approaches? I suspect the answer depends heavily on

data availability and the need for explainability. However, I would like to explore benchmark comparisons in a domain like medical image segmentation where both the stakes and the interpretability requirements are high.

Alexia Y Chavez · ITAI 1378 · Module 04 Image Processing Lab